When Agda met Vampire

Artjoms Šinkarovs and Michael Rawson

Jia-Wei Zhang

10/2026

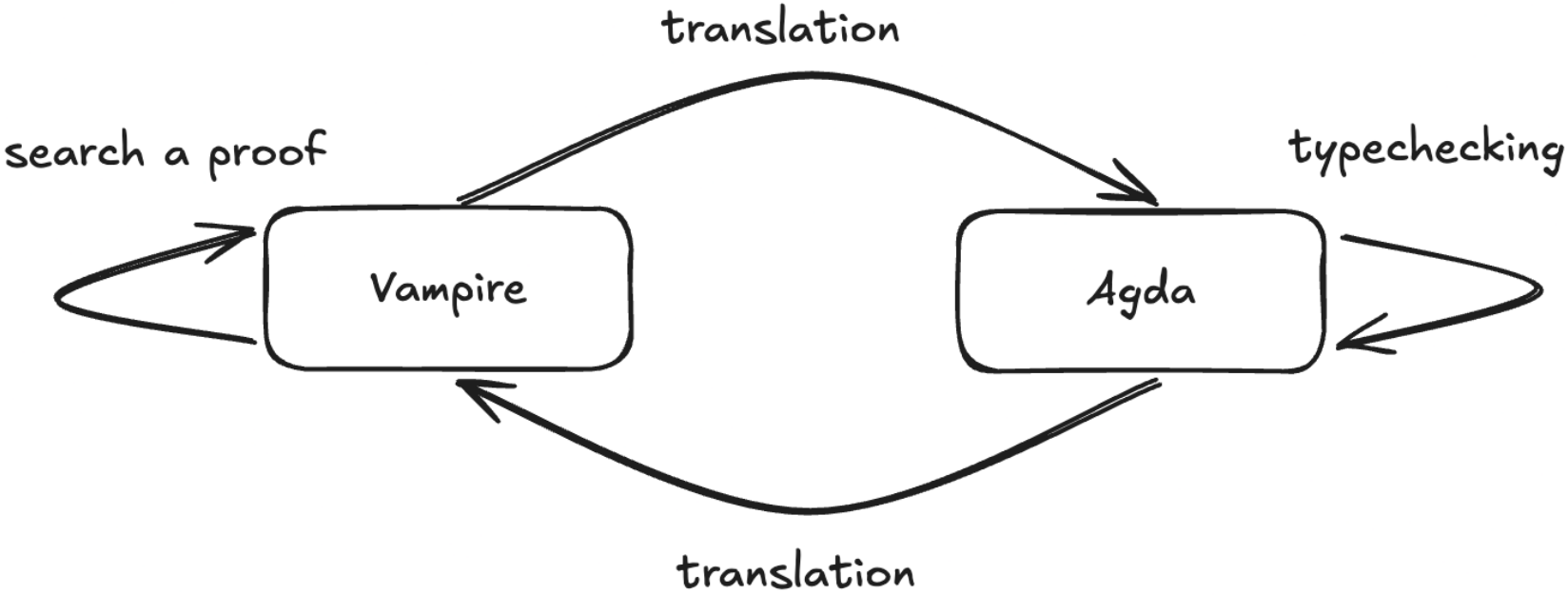
Motivation

Background

ATP	ITP
Automated Theorem Proving ► Machine searches for proof automatically ► ATPs are based on Classical FOL ► Modern ATPs: **E**, **Vampire**	Interactive Theorem Proving ► Human guides proof; machine checks every step ► Most ITPs are based on Intuitionistic Logic ► Proof Assistants: **Agda**, **Rocq**, **Lean**

Vampire, as the one of state-of-the-art ATP, has been under development for nearly 35 years.
Since 1999, Vampire won 78 titles in CASC.
Vampire won all competitions in 2025 and 2026!!

Motivation


translation
search a proof
typechecking
Vampire
Agda
translation

Example 1: Vector Spaces (without Scalars)

```
postulate
  V   : Set                              -- (declare-sort V 0)
  ze  : V                                -- (declare-const ze V)
  _+_ : V → V → V                        -- (declare-fun _+_ (V V) V)
  -_  : V → V                            -- (declare-fun -_ (V) V)

  neutl : ∀ u → ze + u = u               -- (assert (! (forall ((u V))
                                         --   (= (_+_ ze u) u)) :named neutl))
  negl  : ∀ u → (- u) + u = ze           -- (assert (! (forall ((u V))
                                         --   (= (_+_ (-_ u) u) ze))
                                         --   :named negl))
  assoc : ∀ u v w →                      -- (assert (! (forall ((u V)(v V)(w V))
    (u + v) + w = u + (v + w)            --   (= (_+_ (_+_ u v) w)
                                         --      (_+_ u (_+_ v w)))) :named assoc))
                                         -- (declare-const u V)
                                         -- (declare-const v V)
ze-uniq' : ∀ u v →                       -- (assert (! (forall ()
  u + v = v → u = ze                     --   (= (_+_ u v) v)) :named e))
                                         -- (assert-not (! (= u ze) :named ze-uniq))
```

Agda to Vampire is based on **Agda reflection**.

Agda reflection is built-in metaprogramming system.

A Little Taste of Agda

```
trivial : ∀ u → (- u) + (u + u) = u
trivial u = trans (trans t1 t2) (neutl u)
  where
  t1 : (- u) + (u + u) = ((- u) + u) + u
  t1 = sym (assoc (- u) u u)

  t2 : ((- u) + u) + u = ze + u
  t2 = cong (_+ u) (negl u)
```

sym : $\{x\ y : A\} \to x = y \to y = x$
cong : $\{x\ y : A\} \to (f : A \to B) \to x = y \to f\,x = f\,y$
trans : $\{x\ y\ z : A\} \to x = y \to y = z \to x = z$
neutl : $\forall u \to \mathsf{ze} + u = u$
negl : $\forall u \to (-u) + u = \mathsf{ze}$
assoc : $\forall u\ v\ w \to (u + v) + w = u + (v + w)$

Contents

Motivation

Classical FOL

Vampire Core Calculus

Vampire to Agda

Thesis Proposal

References

Classical FOL

Classical FOL

x, y, z as individual variable.

$$
\begin{aligned}
\mathit{term} &::= x \mid c \mid f(\mathit{term}, \ldots, \mathit{term}) \\
\mathit{atom} &::= P(\mathit{term}, \ldots, \mathit{term}) \mid \mathit{term} = \mathit{term} \mid \top \mid \bot \\
\mathit{literal} &::= \mathit{atom} \mid \neg\, \mathit{atom} \\
\varphi &::= \mathit{literal} \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \\
&\quad \mid \varphi \to \varphi \mid \varphi \leftrightarrow \varphi \\
&\quad \mid \forall x\, \varphi \mid \exists x\, \varphi
\end{aligned}
$$

The key point of translation: $P \to Q \equiv \neg P \vee Q$
Vampire is based on **refutation proof**.
(It is **proof by contradiction**.)

Horn Clause and Notation

A **first-order clause** is a $\forall$**-quantified disjunction of literals**.

- **Horn clause**: 0 or 1 positive "head" literals
  - $\forall x_1 \ldots x_k.\ L_1 \vee L_2 \vee \cdots$
- **definite clause**: exactly 1 positive literal
  - $\forall x_1 \ldots x_k.\ \neg A_1 \vee \neg A_2 \vee \cdots \vee B$
  - $\forall x_1 \ldots x_k.\ A_1 \wedge A_2 \wedge \cdots \rightarrow B$
- **goal clause**: exactly 0 positive literals
  - $\forall x_1 \ldots x_k.\ \neg A_1 \vee \neg A_2 \vee \cdots \vee \neg A_n$
  - $\forall x_1 \ldots x_k.\ A_1 \wedge A_2 \wedge \cdots \wedge A_n \rightarrow \bot$

each L_i are Literal and each A_i and B are Atom.
equations $s = t$ are treated as implicitly symmetric.

Vampire Core Calculus

Vampire Core Calculus (Ground and Simplify version)

$$\frac{A \qquad \neg A}{\bot}\ \textsc{Resolution} \qquad \frac{A \vee A}{A}\ \textsc{Factoring}$$

$$\frac{l = r \qquad L[l]}{L[r]}\ \textsc{Replacement} \qquad \frac{s \neq s}{\bot}\ \textsc{Eq. Resolution}$$

A is atom. $L[l], L[r]$ are literals.
l, r, s are terms.

Vampire Core Calculus (Ground)

$$\frac{A \vee C \qquad \neg A \vee D}{C \vee D}\ \textsc{Resolution} \qquad \frac{A \vee A \vee C}{A \vee C}\ \textsc{Factoring}$$

$$\frac{l = r \vee C \qquad L[l] \vee D}{L[r] \vee C \vee D}\ \textsc{Superposition} \qquad \frac{s \neq s \vee C}{C}\ \textsc{Eq. Resolution}$$

A is atom. $L[l], L[r]$ are literals.
l, r, s are terms.

Resolution

$$\dfrac{\dfrac{A \vee C}{\neg A \to C} \qquad \dfrac{\neg A \vee D}{A \to D}}{C \vee D}\ \textsc{Res.}$$

-

- If A is true, then D.
- If A is false, then C.

Either way, $C \vee D$ holds, right?

A is atom. C, D are Clauses.

Resolution produces an equisatisfiable clause

A	C	D	$(A \vee C) \wedge (\neg A \vee D)$	$C \vee D$
T	T	T	T	T
T	T	F	F	T
T	F	T	T	T
T	F	F	F	F
F	T	T	T	T
F	T	F	T	T
F	F	T	F	T
F	F	F	F	F

Equality Resolution

$$\frac{s = s \qquad s \neq s}{\bot}$$

$$\downarrow \text{ add clauses}$$

$$\frac{s = s \qquad s \neq s \vee C}{C} \xrightarrow{A{:}s{=}s;\ \neg A{:}s{\neq}s} \frac{\neg A \vee C \qquad A}{C}\ \textsc{Res.}$$

$$\updownarrow$$

$$\frac{A \to C \qquad A}{C}\ \text{MP.}$$

Modus Ponens is special case of Resolution in Classical Logic.

Superposition

$$\frac{l = r \vee C \qquad L[l] \vee D}{L[r] \vee C \vee D}\ \text{SUP.}$$

-

Because $l = r$ is true and $L[l]$.

$$C \wedge (L[r] \vee D)$$

$$(C \wedge L[r]) \vee (C \wedge D)$$

- If C is true, then $L[r] \vee D$.
- If C is false, then formulae is false.

Either way, $L[r] \vee C \vee D$ holds.

Vampire Core Calculus (Nonground)

$$\frac{A \vee C \qquad \neg A' \vee D}{\sigma(C \vee D)}\ \textsc{Resolution} \qquad \frac{A \vee A' \vee C}{\sigma(A \vee C)}\ \textsc{Factoring}$$

$$\frac{l = r \vee C \qquad L[l'] \vee D}{\sigma(L[r] \vee C \vee D)}\ \textsc{Superposition} \qquad \frac{s \neq t \vee C}{\sigma(C)}\ \textsc{Eq. Resolution}$$

$\sigma : \mathit{Var} \to \mathit{Term}$
$\mathrm{mgu}(A, A')$ in Resolution and Factoring.
$\mathrm{mgu}(l, l')$ in Superposition.
$\mathrm{mgu}(s, t)$ in Equality Resolution.
A is atom. $L[l'], L[r]$ are literals.
C, D are clauses.
l, l', r, s, t are terms.

Resolution (Nonground)

$$\dfrac{\dfrac{\dfrac{A \vee C}{\sigma(\neg A \to C)}}{\neg\sigma(A) \to \sigma(C)} \qquad \dfrac{\dfrac{\neg A' \vee D}{\sigma(A' \to D)}}{\sigma(A') \to \sigma(D)}}{\sigma(C \vee D)}\ \textsc{Res.}$$

- If $\sigma(A')$ is true, then $\sigma(D)$.
- If $\sigma(A)$ is false, then $\sigma(C)$.

Either way, $\sigma(C) \vee \sigma(D)$ holds, $\sigma(C \vee D)$ also holds.

A is atom. C, D are Clauses.
$\sigma = mgu(A, A')$, $\sigma(A) = \sigma(A')$

Equality Resolution (Nonground)

$$\frac{\dfrac{s \neq t \vee C}{\sigma(s \neq t) \vee \sigma(C)} \qquad \sigma(s) = \sigma(t)}{\sigma(s) \neq \sigma(t) \vee \sigma(C)} \qquad \frac{}{\sigma(C)}\ \text{Eq.Res}$$

$\sigma = mgu(s, t)$, $\sigma(s) = \sigma(t)$

Superposition (Nonground)

$$\frac{l = r \vee C \qquad L[l'] \vee D}{\sigma(l = r) \vee \sigma(C) \qquad \sigma(L[l']) \vee \sigma(D)} \text{ SUP.}$$

$$\sigma(L[r] \vee C \vee D)$$

Because $\sigma(l) = \sigma(r)$ is true and $\sigma(L[\sigma(l')])$.

$$\sigma(C) \wedge (\sigma(L[r]) \vee \sigma(D))$$

$$(\sigma(C) \wedge \sigma(L[r])) \vee (\sigma(C) \wedge \sigma(D))$$

- If $\sigma(C)$ is true, then $\sigma(L[r]) \vee \sigma(D)$.
- If $\sigma(C)$ is false, then formulae is false.

Either way, $\sigma(L[r]) \vee \sigma(C) \vee \sigma(D)$ holds.

$\sigma = mgu(l, l')$, $\sigma(l) = \sigma(l')$; $\sigma(L[l']) = \sigma(L[\sigma(l')])$

Example 1 (Vector Spaces)

```
 1. ∀x. ze + x = x  (neutl)
 2. ∀x. (-x) + x = ze  (negl)
 3. ∀x y z. (x + y) + z = x + (y + z)  (assoc)
 4. v = u + v  (e)
 5. ze != u  (negated goal)
 6. ∀x y. (-x) + (x + y) = ze + y  (superposition 3, 2)
 7. ∀x. v + x = u + (v + x)  (superposition 3, 4)
 8. ∀x y. (-x) + (x + y) = y  (superposition 6, 1)
 9. ∀x. (-(-x)) + ze = x  (superposition 8, 2)
10. ∀x y. y + x = (-(-y)) + x  (superposition 8, 8)
11. ∀x. x + ze = x  (superposition 9, 10)
12. ∀x. ze = x + (-x)  (superposition 2, 10)
13. ze = u + ze  (superposition 7, 12)
14. ze = u  (superposition 13, 11)
15. false  (resolution 14, 5)

-- Vampire Original Output
 1. 'vec._+_'('vec.ze',X0) = X0
 2. 'vec._+_'('vec.-_'(X0),X0) = 'vec.ze'
 3. 'vec._+_'('vec._+_'(X0,X1),X2) = 'vec._+_'(X0,'vec._+_'(X1,X2))
 4. v = 'vec._+_'(u,v)
 5. 'vec.ze' != u
 6. 'vec._+_'('vec.-_'(X0),'vec._+_'(X0,X1)) = 'vec._+_'('vec.ze',X1)
 7. 'vec._+_'(v,X0) = 'vec._+_'(u,'vec._+_'(v,X0))
 8. 'vec._+_'('vec.-_'(X0),'vec._+_'(X0,X1)) = X1
 9. 'vec._+_'('vec.-_'('vec.-_'(X0)),'vec.ze') = X0
10. 'vec._+_'(X1,X0) = 'vec._+_'('vec.-_'('vec.-_'(X1)),X0)
11. 'vec._+_'(X0,'vec.ze') = X0
12. 'vec.ze' = 'vec._+_'(X0,'vec.-_'(X0))
13. 'vec.ze' = 'vec._+_'(u,'vec.ze')
14. 'vec.ze' = u
15. $false
```

Example 1 (Vector Spaces)

$$
\begin{array}{lll}
2 & \forall x.\ (-x) + x = \mathsf{ze} & (l = r) \\
3 & \forall x\, y\, z.\ (x + y) + z = x + (y + z) & (L[l'])
\end{array}
$$

Sup. 3, 2

$$\frac{l = r \qquad L[l']}{\sigma(L[r])}\ \textsc{Sup.}$$

$$\sigma = \{\, x \mapsto -x,\ y \mapsto x \,\}$$

$$\frac{(-x) + x = \mathsf{ze} \qquad ((-x) + x) + z = (-x) + (x + z)}{\mathsf{ze} + z = (-x) + (x + z)}\ \textsc{Sup.}$$

Vampire to Agda

A classical proof to intuitionistic proof

We restrict Vampire to Horn Clauses and show $\Gamma \vdash G$.
Assuming that this is provable, and Vampire obtains a classical proof by refutation showing $\Gamma, \neg G \vdash \bot$

- $\Gamma, \neg G \vdash \bot$
- $\Gamma, G \to \bot \vdash \bot$
- $\Gamma, G \to G \vdash G$ (replacing occurrences of $\bot$ with G)

Γ is “a set of definite clauses”, G is “ground goal atom”.

Lemmas

- Lemma 1: For every VCC rule, if its premises are Horn, then its conclusion is also Horn.
  (Translate Horn Clause to Horn Implication.)
- Lemma 2: Horn Vampire Core Calculus (HVCC) is intuitionistically valid.
- Lemma 3: Let Γ be definite clauses and G **be a ground atom.** If $\Gamma, G \to \bot \vdash \bot$ is derivable in HVCC, then $\Gamma, G \to G \vdash G$ is also derivable in HVCC.

Vampire Core Calculus = VCC, Horn VCC = HVCC

Proof of Lemma 1

VCC

$$\frac{A \vee C \quad \neg A' \vee D}{\sigma(C \vee D)}\ \textsc{Res.}$$

$$\frac{A \vee A' \vee C}{\sigma(A \vee C)}\ \textsc{Fac.}$$

$$\frac{s \neq t \vee C}{\sigma(C)}\ \textsc{Eq.Res.}$$

$$\frac{l = r \vee C \quad L[l'] \vee D}{\sigma(L[r] \vee C \vee D)}\ \textsc{Sup.}$$

$$\frac{\neg P \vee A \quad \neg A' \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg Q \vee \boxed{B})}\ \textsc{Res.}$$

$$\frac{\neg A \vee \neg A' \vee \neg P \vee \boxed{B}}{\sigma(\neg A \vee \neg P \vee \boxed{B})}\ \textsc{Fac.}$$

$$\frac{s \neq t \vee \neg P \vee \boxed{A}}{\sigma(\neg P \vee \boxed{A})}\ \textsc{Eq.Res.}$$

$$\frac{\neg P \vee l = r \quad \neg A[l'] \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg A[r] \vee \neg Q \vee \boxed{B})}\ \textsc{Sup.L}$$

$$\frac{\neg P \vee l = r \quad \neg Q \vee \boxed{A[l']}}{\sigma(\neg P \vee \neg Q \vee \boxed{A[r]})}\ \textsc{Sup.R}$$

P, Q = premises; A, B = atoms; C, D = clauses; $L[l']$, $L[r]$ = literals;
$\mathrm{mgu}(A, A')$, $\mathrm{mgu}(l, l')$, $\mathrm{mgu}(s, t)$
$\boxed{\ }$ = positive atom.
$\neg P = \neg(A_1 \wedge A_2 \wedge \cdots)$, $C = \neg A_1 \vee \neg A_2 \vee \cdots$ (Horn and De Morgan)

Horn Vampire Core Calculus (HVCC)

$$\frac{\neg P \vee A \quad \neg A' \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg Q \vee \boxed{B})}\ \textsc{Res.}$$

$$\frac{\neg A \vee \neg A' \vee \neg P \vee \boxed{B}}{\sigma(\neg A \vee \neg P \vee \boxed{B})}\ \textsc{Fac.}$$

$$\frac{s \not\approx t \vee \neg P \vee \boxed{A}}{\sigma(\neg P \vee \boxed{A})}\ \textsc{Eq.Res.}$$

$$\frac{\neg P \vee l = r \quad \neg A[l'] \vee \neg Q \vee \boxed{B}}{\sigma(\neg P \vee \neg A[r] \vee \neg Q \vee \boxed{B})}\ \textsc{Sup.L}$$

$$\frac{\neg P \vee l = r \quad \neg Q \vee \boxed{A[l']}}{\sigma(\neg P \vee \neg Q \vee \boxed{A[r]})}\ \textsc{Sup.R}$$

HVCC

$$\frac{P \to A \quad A' \wedge Q \to \boxed{B}}{\sigma(P \wedge Q \to \boxed{B})}\ \textsc{Res.}$$

$$\frac{A \wedge A' \wedge P \to \boxed{B}}{\sigma(A \wedge P \to \boxed{B})}\ \textsc{Fac.}$$

$$\frac{s \not\approx t \wedge P \to \boxed{A}}{\sigma(P \to \boxed{A})}\ \textsc{Eq.Res.}$$

$$\frac{P \to l = r \quad A[l'] \wedge Q \to \boxed{B}}{\sigma(P \wedge A[r] \wedge Q \to \boxed{B})}\ \textsc{Sup.L}$$

$$\frac{P \to l = r \quad Q \to \boxed{A[l']}}{\sigma(P \wedge Q \to \boxed{A[r]})}\ \textsc{Sup.R}$$

P, Q = premises; A, B = atoms; $\mathrm{mgu}(A, A')$, $\mathrm{mgu}(l, l')$, $\mathrm{mgu}(s, t)$

$\neg P = \neg(A_1 \wedge A_2 \wedge \cdots) \Longrightarrow \neg A_1 \vee \neg A_2 \vee \cdots \neg A_n$

definite clause: $\neg A_1 \vee \neg A_2 \vee \cdots \vee B \Longrightarrow A_1 \wedge A_2 \wedge \cdots \to B$

Proof of Lemma 2

HVCC

$$\frac{P \to A \quad A' \wedge Q \to \boxed{B}}{\sigma(P \wedge Q \to \boxed{B})}\ \textsc{Res.}$$

$$\frac{A \wedge A' \wedge P \to \boxed{B}}{\sigma(A \wedge P \to \boxed{B})}\ \textsc{Fac.}$$

$$\frac{s = t \wedge P \to \boxed{A}}{\sigma(P \to \boxed{A})}\ \textsc{Eq.Res.}$$

$$\frac{P \to l = r \quad A[l'] \wedge Q \to \boxed{B}}{\sigma(P \wedge A[r] \wedge Q \to \boxed{B})}\ \textsc{Sup.L}$$

$$\frac{P \to l = r \quad Q \to \boxed{A[l']}}{\sigma(P \wedge Q \to \boxed{A[r]})}\ \textsc{Sup.R}$$

Agda

$$\frac{t_1 : P \to A \quad t_2 : A \wedge Q \to B}{(\lambda(p, q).\ t_2\ (t_1\ p, q)) : P \wedge Q \to B}\ \textsc{Res.}$$

$$\frac{t : A \wedge A \wedge P \to B}{(\lambda(a, p).\ t\ (a, a, p)) : A \wedge P \to B}\ \textsc{Fac.}$$

$$\frac{t : s = s \wedge P \to A}{(\lambda p.\ t\ \mathsf{refl}\ p) : P \to A}\ \textsc{Eq.Res.}$$

$$\frac{t_1 : P \to l = r \quad t_2 : A[l] \wedge Q \to B}{\begin{array}{c}(\lambda(p, a, q).\ t_2\ ((\mathsf{subst}\ (\mathsf{sym}\ (t_1\ p))\ a), q)) \\ : P \wedge A[r] \wedge Q \to B\end{array}}\ \textsc{Sup.L}$$

$$\frac{t_1 : P \to l = r \quad t_2 : Q \to A[l]}{(\lambda(p, q).\ \mathsf{subst}\ (t_1\ p)\ (t_2\ q)) : P \wedge Q \to A[r]}\ \textsc{Sup.R}$$

refl : $l = l$, sym : $l = r \to r = l$, subst : $l = r \to A[l] \to A[r]$.

An example of applying Lemma 3

$$t_1 : \forall x.\, f(x) = x,\quad t_2 : a = f(c),\quad t_3 : b = c,\quad t_4 : a = b \to \boxed{\bot} \;\vdash\; \boxed{\bot}$$

$$\dfrac{b = c \qquad \dfrac{f(c) = c \qquad \dfrac{a = f(c) \qquad a = b \to \boxed{\bot}}{f(c) = b \to \boxed{\bot}}\ \textsc{Sup.L}}{c = b \to \boxed{\bot}}\ \textsc{Sup.L}}{c = c \to \boxed{\bot}}\ \textsc{Sup.L}$$

$$\dfrac{c = c \to \boxed{\bot}}{\boxed{\bot}}\ \textsc{Eq.Res.}$$

replace each $\boxed{\bot}$ by $a = b$

Thesis Proposal

Thesis Proposal

Formalize and verify the translation from Vampire to Agda.

- Formalize HVCC by deep embedding.
- Prove Lemmas 2, 3.

References

- Artjoms Šinkarovs and Michael Rawson. “When Agda met Vampire.”
- Laura Kovács and Andrei Voronkov. “Vampire Guide Lectures.”
- Giles Reger. “The Vampire Journey. (Youtube)”
- Alan J.A. Robinson, Andrei Voronkov. “Paramodulation-Based Theorem Proving”